

Figure 8: Sample of successful POST operation

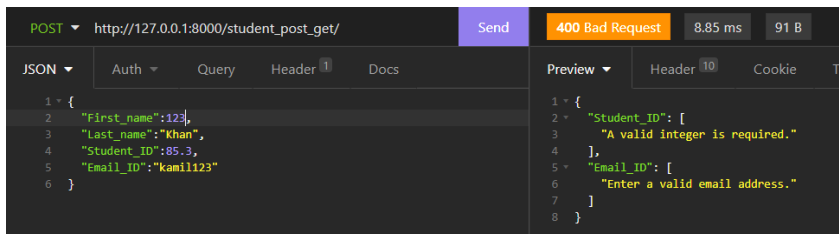


Figure 9: Sample error messages

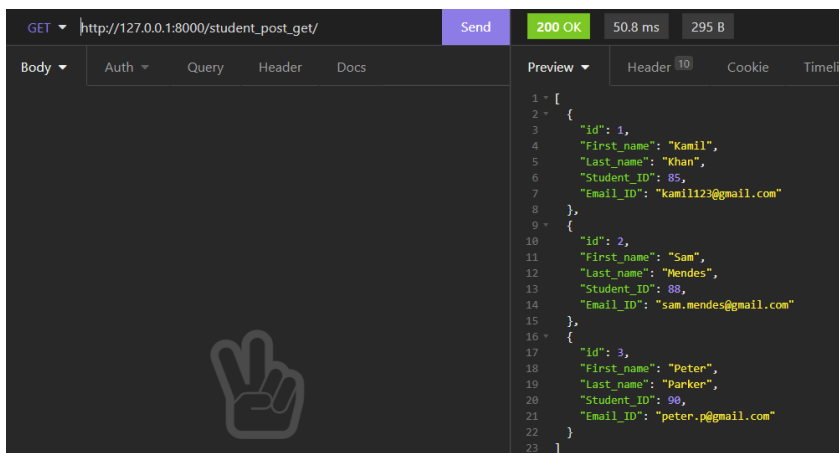


Figure 10: Displaying all information using GET

```
data,status=status.HTTP_201_CREATED)
    return Response(serializer.
errors,status=status.HTTP_400_BAD_
REQUEST)

def get(self,request):
    ---(15)
    getting_data=Studentinfo.
objects.all()
    serializer=Studentserializer(get
tting_data,many=True)
    return Response(serializer.
data)
```

The function 'post' corresponds to 'Create' operation, while the 'get' function aligns with the 'Read' operation. In line (9), the 'request.

data' is a representation of the data we 'POST' in JSON format. 'request.data' being assigned to the 'data' parameter within the Studentserializer implies that the entered data is passed to the serializer. Line (10) checks the validation of the data entered for the constituting fields within the model that the 'Studentserializer' is associated with, i.e., Studentinfo. Line (12) displays the entered data itself, indicating that there are no validation errors, post the saving mentioned in line (11). Line (13) helps return the errors (if any). As per line (16), all the objects present within the 'Studentinfo' model (the existing data) are assigned to

'getting_data' variable. These objects are then passed through the serializer to be presented in JSON format as per line (18). Just one step remains before we can initiate inserting values for our table. In 'urls.py' within 'osfproject1', insert the following:

```
from django.contrib import admin
from django.urls import path
from firstapp.views import Student_post_
get_view

urlpatterns = [
    ---(5)
    path('admin/', admin.site.urls),
    path('student_post_get/',Student_
post_get_view.as_view()),
]
```

The URL path we entered in line (7) helps us in carrying out the operations present within 'Student_post_get_view', i.e., the 'POST' and 'GET'. In the prompt, type:

```
python manage.py runserver
```

You should be able to see the link <http://127.0.0.1:8000/> in the output, implying that the server is running. We can now head to Insomnia to carry out the 'POST' and 'GET' operations (Figure 8)!

Now let's try botching these up to see how validation errors are displayed.

As observed in Figure 9, we entered incorrect types of values for 'First_name', 'Student_ID' and 'Email_ID' fields, but why is it that we got error messages for only two of them? This is because a serializer takes the values that you enter for the fields and tries converting them to the field types you defined in the model, and checks for successful conversion. Generally, in Python, a number of any type — be it float, string, etc — can be converted to a string. Since it is possible for the number 123 to be converted to string in Python, no error message is displayed for the same. We will enter valid data for two more students and then perform the 'GET'